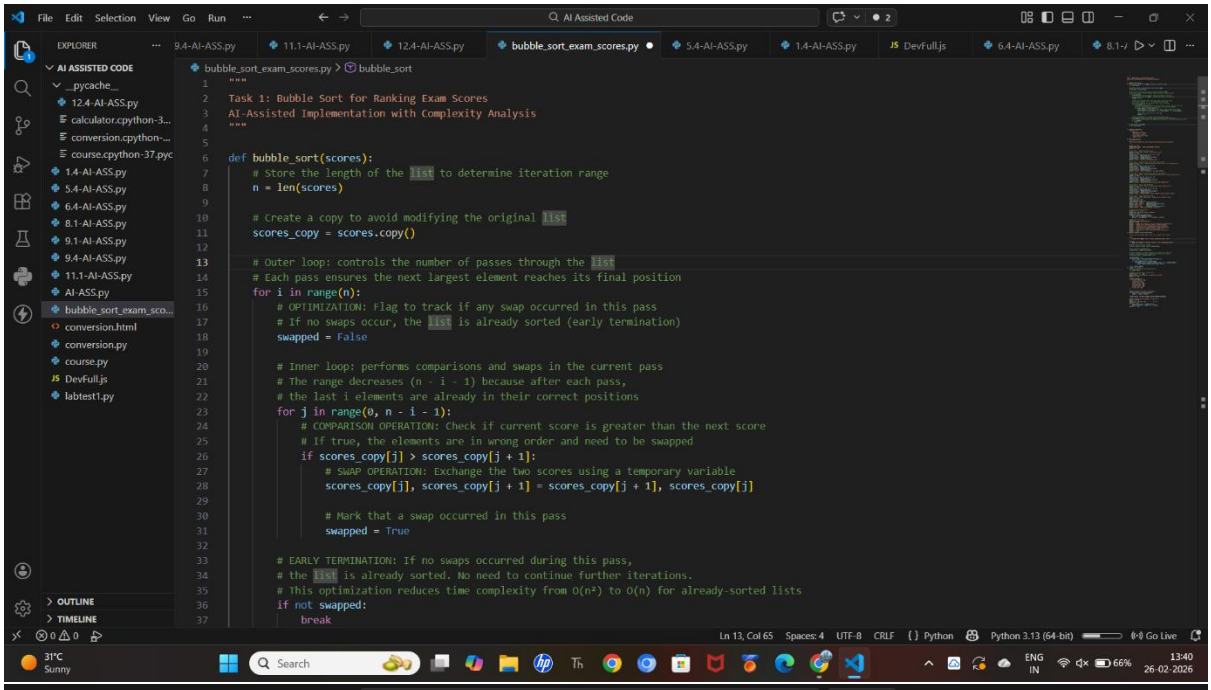
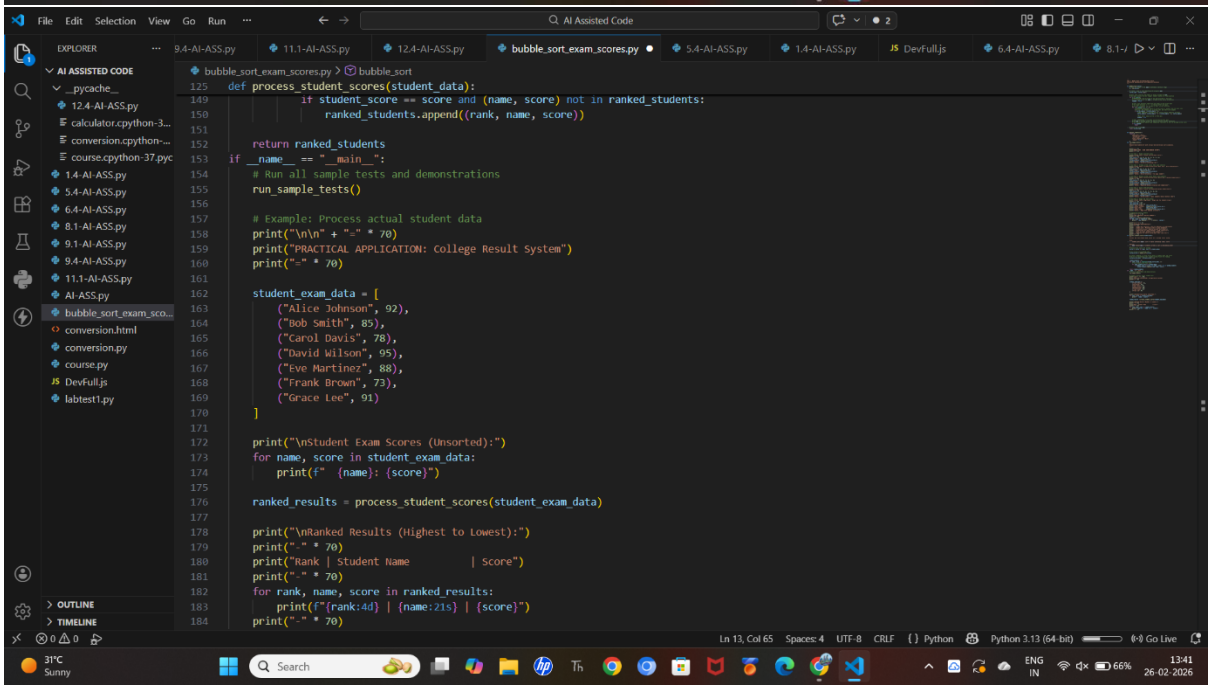
12.4-AI-ASSIGNMENT
2303A52249
A.VAMSHI KRISHNA

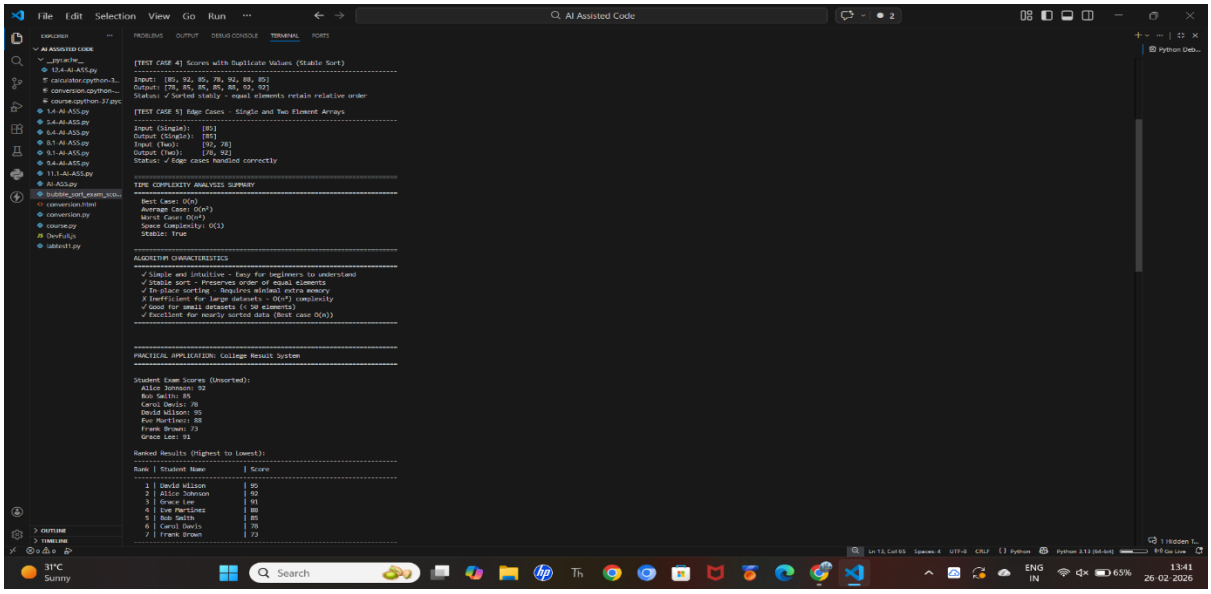
TASK-1
Scenario:
You are building a college result system where a small list of student exam scores needs to be sorted after each internal assessment to calculate rankings.
Prompt:
“Write a Python program for Bubble Sort to sort student scores. Add inline comments explaining comparisons and swaps. Include early termination optimization and explain time complexity.”
Code:



```python
"""
Task 1: Bubble Sort for Ranking Exam Scores
AI-Assisted Implementation with Complexity Analysis
"""

def bubble_sort(scores):
    # Store the length of the list to determine iteration range
    n = len(scores)

    # Create a copy to avoid modifying the original list
    scores_copy = scores.copy()

    # Outer loop: controls the number of passes through the list
    # Each pass ensures the next largest element reaches its final position
    for i in range(n):
        # OPTIMIZATION: Flag to track if any swap occurred in this pass
        # If no swaps occur, the list is already sorted (early termination)
        swapped = False

        # Inner loop: performs comparisons and swaps in the current pass
        # The range decreases (n - i - 1) because after each pass,
        # the last i elements are already in their correct positions
        for j in range(0, n - i - 1):
            # COMPARISON OPERATION: Check if current score is greater than the next score
            # If true, the elements are in wrong order and need to be swapped
            if scores_copy[j] > scores_copy[j + 1]:
                # SWAP OPERATION: Exchange the two scores using a temporary variable
                scores_copy[j], scores_copy[j + 1] = scores_copy[j + 1], scores_copy[j]

                # Mark that a swap occurred in this pass
                swapped = True

        # EARLY TERMINATION: If no swaps occurred during this pass,
        # the list is already sorted. No need to continue further iterations.
        # This optimization reduces time complexity from O(n²) to O(n) for already-sorted lists
        if not swapped:
            break
```



```python
def process_student_scores(student_data):
            if student_score == score and (name, score) not in ranked_students:
                ranked_students.append((rank, name, score))

    return ranked_students
if __name__ == "__main__":
    # Run all sample tests and demonstrations
    run_sample_tests()

    # Example: Process actual student data
    print("\n\n" + "=" * 70)
    print("PRACTICAL APPLICATION: College Result System")
    print("=" * 70)

    student_exam_data = [
        ("Alice Johnson", 92),
        ("Bob Smith", 85),
        ("Carol Davis", 78),
        ("David Wilson", 95),
        ("Eve Martinez", 88),
        ("Frank Brown", 73),
        ("Grace Lee", 91)
    ]

    print("\nStudent Exam Scores (Unsorted):")
    for name, score in student_exam_data:
        print(f"  {name}: {score}")

    ranked_results = process_student_scores(student_exam_data)

    print("\nRanked Results (Highest to Lowest):")
    print("-" * 70)
    print("Rank | Student Name          | Score")
    print("-" * 70)
    for rank, name, score in ranked_results:
        print(f"{rank:4d} | {name:21s} | {score}")
    print("-" * 70)
```



Observation:

AI generated clear code with comments and Used a `swapped` flag for early stopping (optimization).
Time Complexity:

- Best Case: **O(n)**
- Average Case: **O(n²)**
- Worst Case: **O(n²)**

- Suitable for small datasets, not efficient for large data.

TASK-2

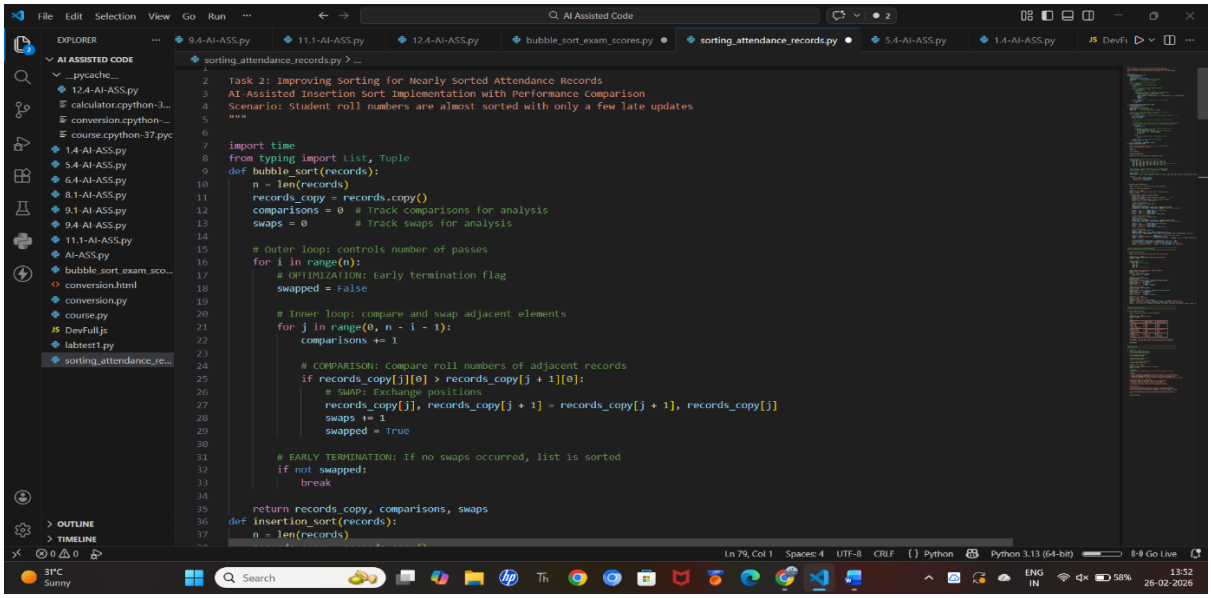
Scenario:

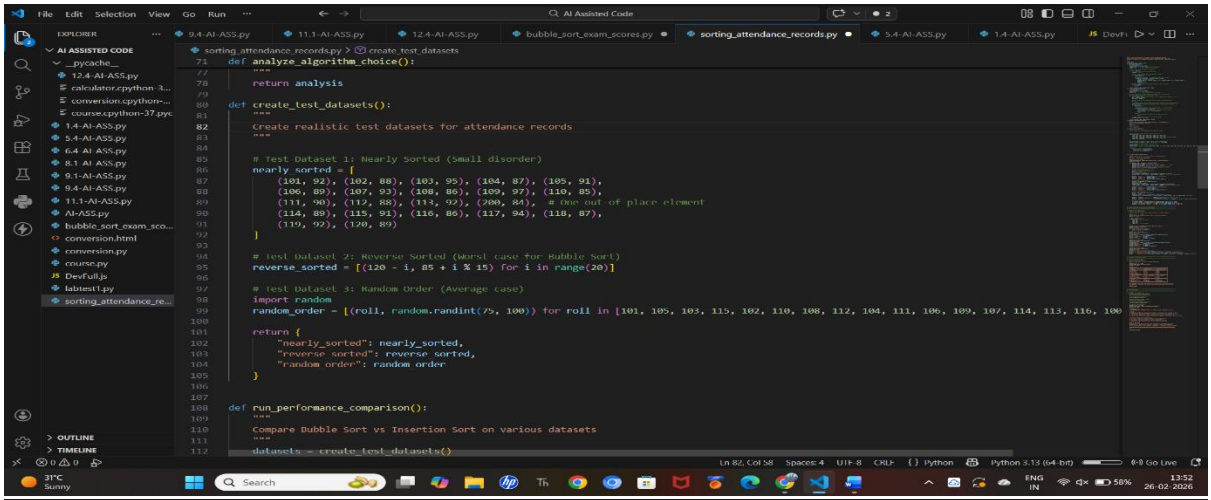
You are maintaining a student attendance system where roll numbers are almost sorted, with only a few late updates or changes.

Prompt:

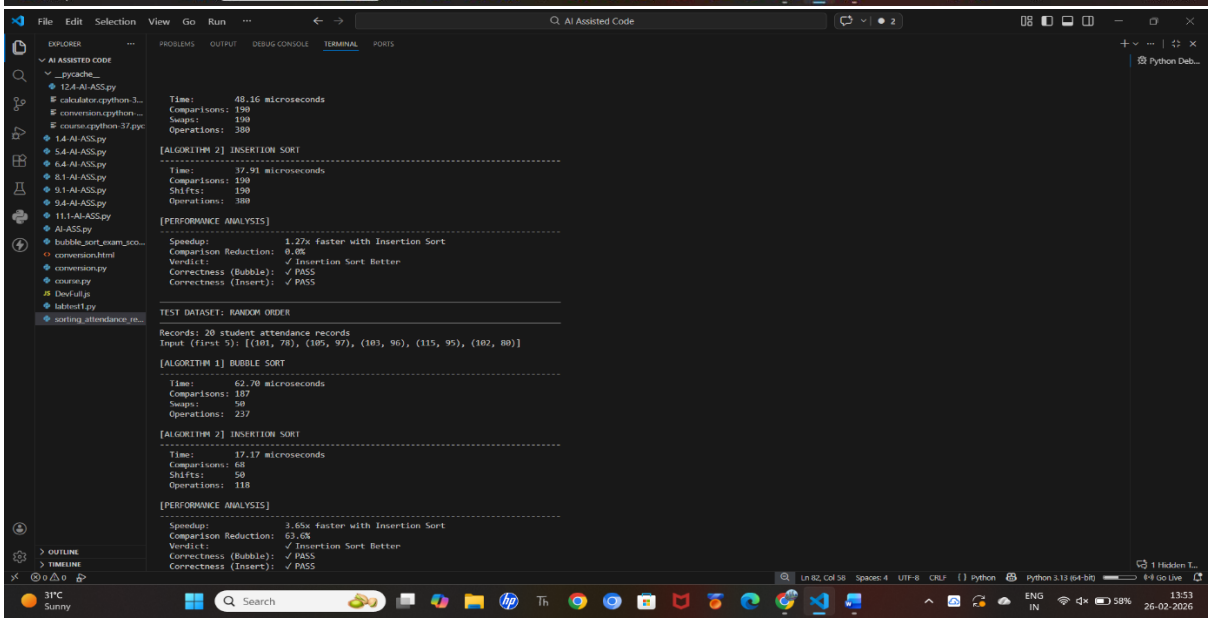
“Review this problem of nearly sorted attendance records. Suggest a better sorting algorithm than Bubble Sort. Generate an Insertion Sort implementation and explain why it performs better on nearly sorted data. Compare execution behavior.”

Code:





```
    def analyze_algorithm_choice():
        """
        return analysis

def create_test_datasets():
    """
    Create realistic test datasets for attendance records
    """

    # Test Dataset 1: Nearly Sorted (Small disorder)
    nearly_sorted = [
        (101, 92), (102, 88), (103, 95), (104, 87), (105, 91),
        (106, 89), (107, 93), (108, 86), (109, 97), (110, 85),
        (111, 90), (112, 88), (113, 92), (200, 84),  # One out of place element
        (114, 89), (115, 91), (116, 86), (117, 94), (118, 87),
        (119, 92), (120, 89)
    ]

    # Test Dataset 2: Reverse Sorted (Worst case for Bubble Sort)
    reverse_sorted = [(120 - i, 85 + i % 15) for i in range(20)]

    # Test Dataset 3: Random Order (Average case)
    import random
    random_order = [(roll, random.randint(75, 100)) for roll in [101, 105, 103, 115, 102, 110, 108, 112, 104, 111, 106, 109, 107, 114, 113, 116, 100

    return {
        "nearly_sorted": nearly_sorted,
        "reverse_sorted": reverse_sorted,
        "random_order": random_order
    }


def run_performance_comparison():
    """
    Compare Bubble Sort vs Insertion Sort on various datasets
    """
    datasets = create_test_datasets()
```



```
Time:       48.16 microseconds
Comparisons: 190
Swaps:      190
Operations:  380

[ALGORITHM 2] INSERTION SORT
------------------------------------------------------------------------------
Time:       37.91 microseconds
Comparisons: 190
Shifts:     190
Operations:  380

[PERFORMANCE ANALYSIS]
------------------------------------------------------------------------------
Speedup:              1.27x faster with Insertion Sort
Comparison Reduction: 0.0%
Verdict:              ✓ Insertion Sort Better
Correctness (Bubble): ✓ PASS
Correctness (Insert): ✓ PASS

================================================================================
TEST DATASET: RANDOM ORDER
================================================================================
Records: 20 student attendance records
Input (first 5): [(101, 78), (105, 97), (103, 96), (115, 95), (102, 80)]

[ALGORITHM 1] BUBBLE SORT
------------------------------------------------------------------------------
Time:       62.70 microseconds
Comparisons: 187
Swaps:      50
Operations:  237

[ALGORITHM 2] INSERTION SORT
------------------------------------------------------------------------------
Time:       17.17 microseconds
Comparisons: 68
Shifts:     50
Operations:  118

[PERFORMANCE ANALYSIS]
------------------------------------------------------------------------------
Speedup:              3.65x faster with Insertion Sort
Comparison Reduction: 63.6%
Verdict:              ✓ Insertion Sort Better
Correctness (Bubble): ✓ PASS
Correctness (Insert): ✓ PASS
```

Observation:

AI suggested Insertion Sort instead of Bubble Sort.
Reason: Insertion Sort works efficiently when data is already nearly sorted.
It shifts elements instead of repeatedly swapping like Bubble Sort.
In nearly sorted data, Insertion Sort performs closer to O(n).
Bubble Sort still makes many unnecessary comparisons.
Therefore, Insertion Sort is more suitable for attendance records.

TASK-3

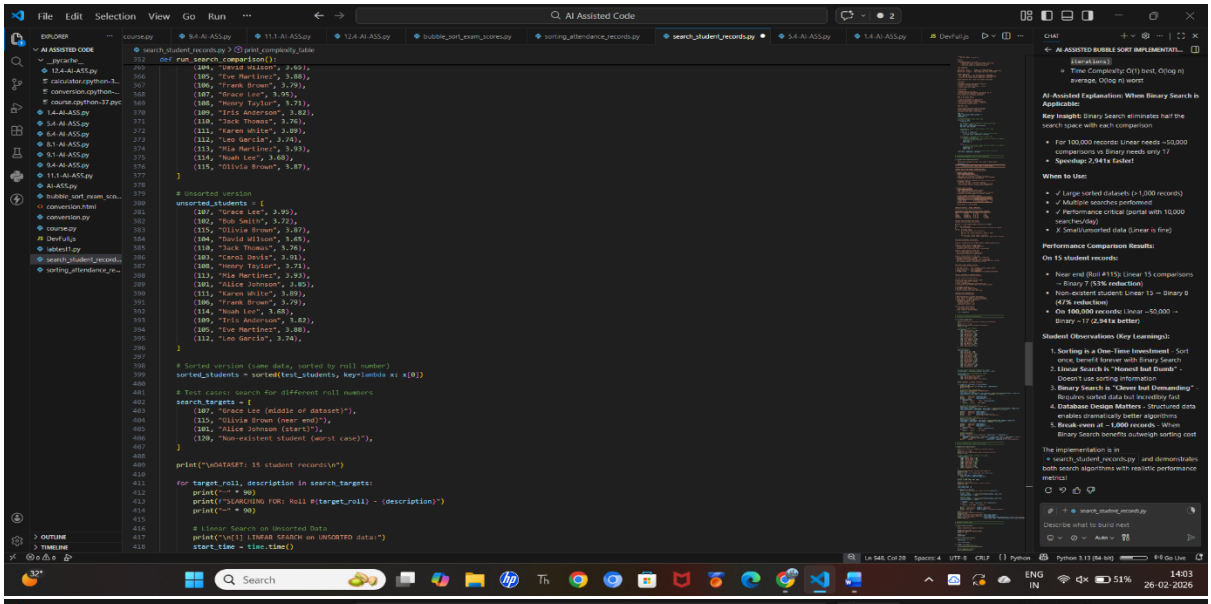
Scenario:

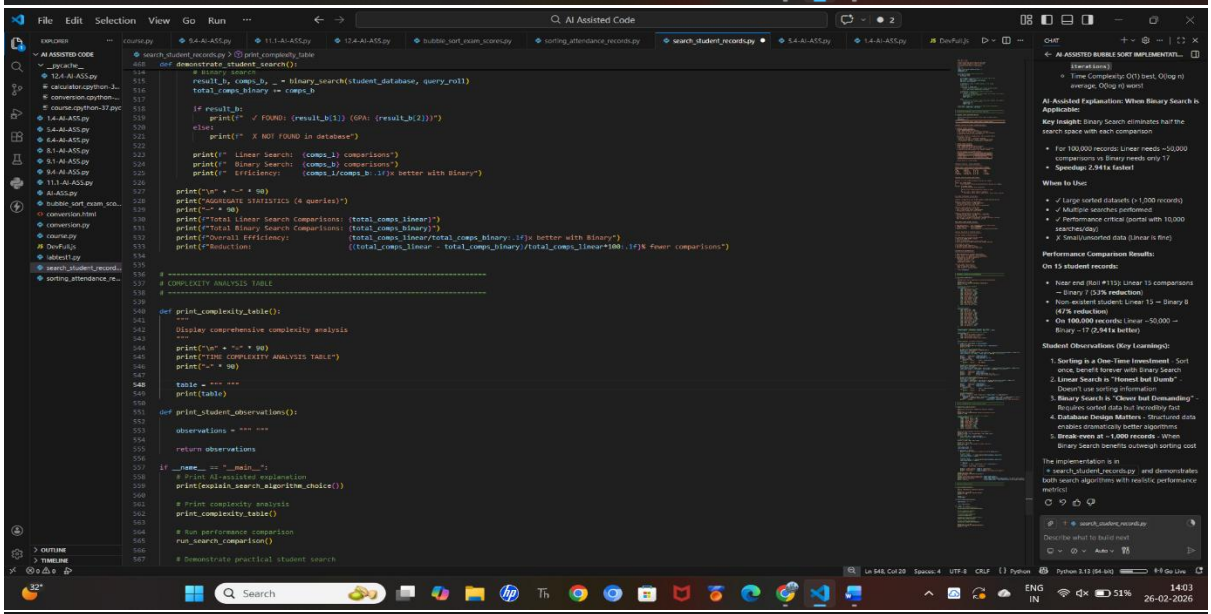
You are building a student information portal where users search for student records using roll numbers. The data may be unsorted or sorted.

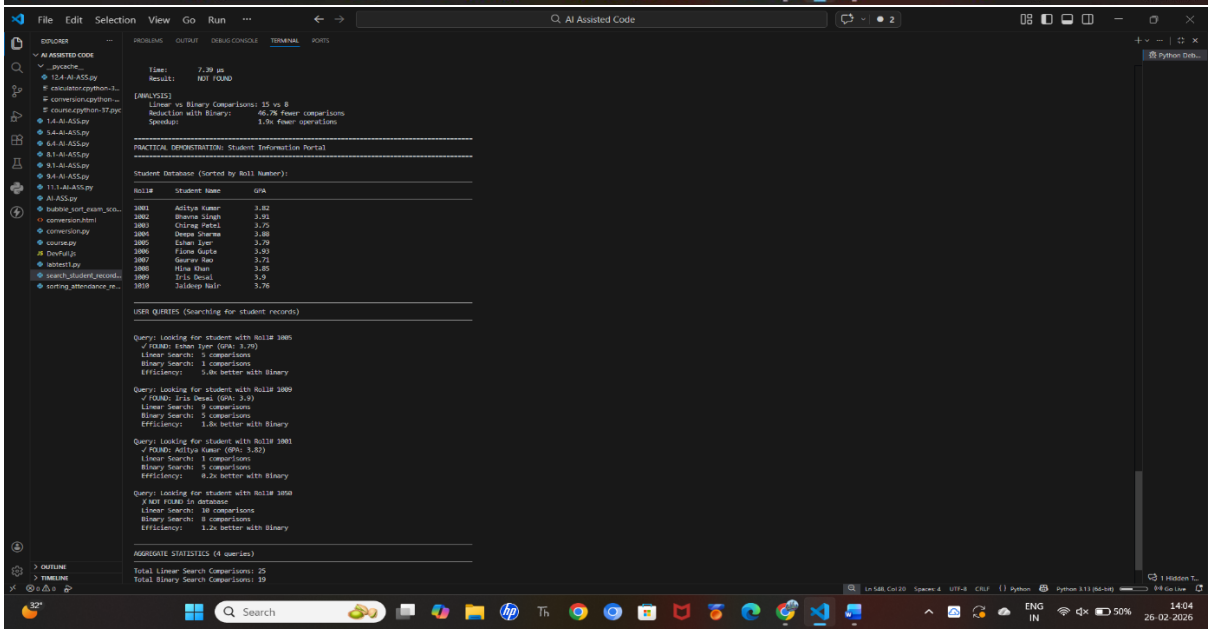
Prompt:

“Write Python programs for Linear Search (unsorted data) and Binary Search (sorted data). Add proper docstrings explaining parameters and return values. Explain when Binary Search can be used and compare performance differences.”

Code:







Observation:

AI generated both search implementations with clear docstrings.
Linear Search checks each element one by one.
Binary Search repeatedly divides the sorted list into halves.

Algorithm	Best Case	Worst Case
Linear Search	O(1)	O(n)
Binary Search	O(1)	O(log n)

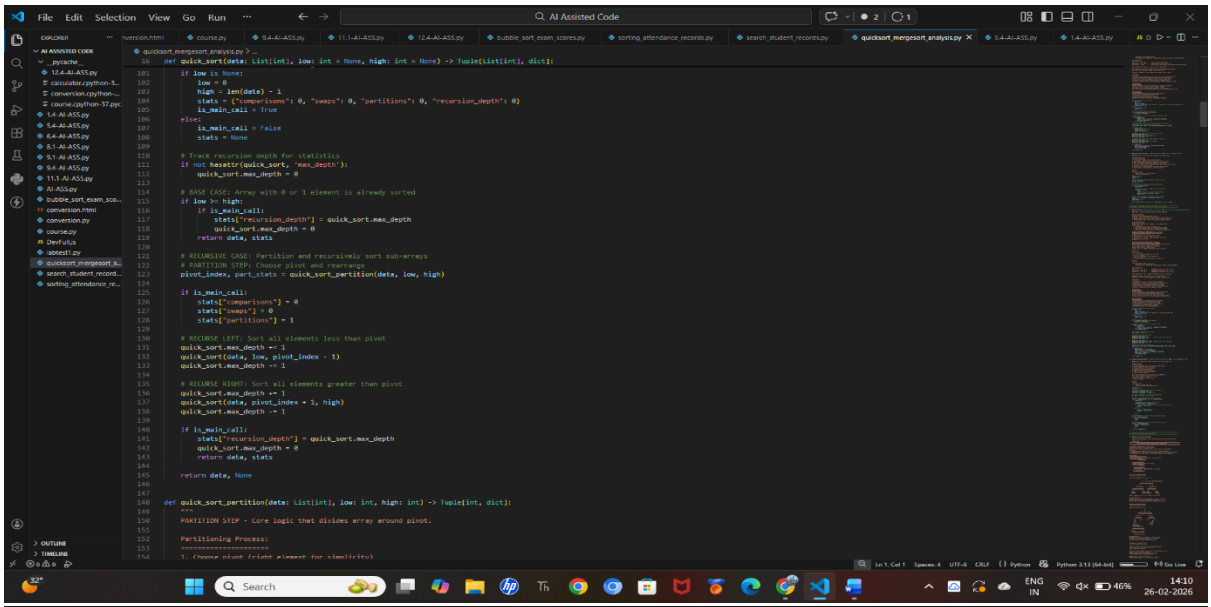
TASK-4

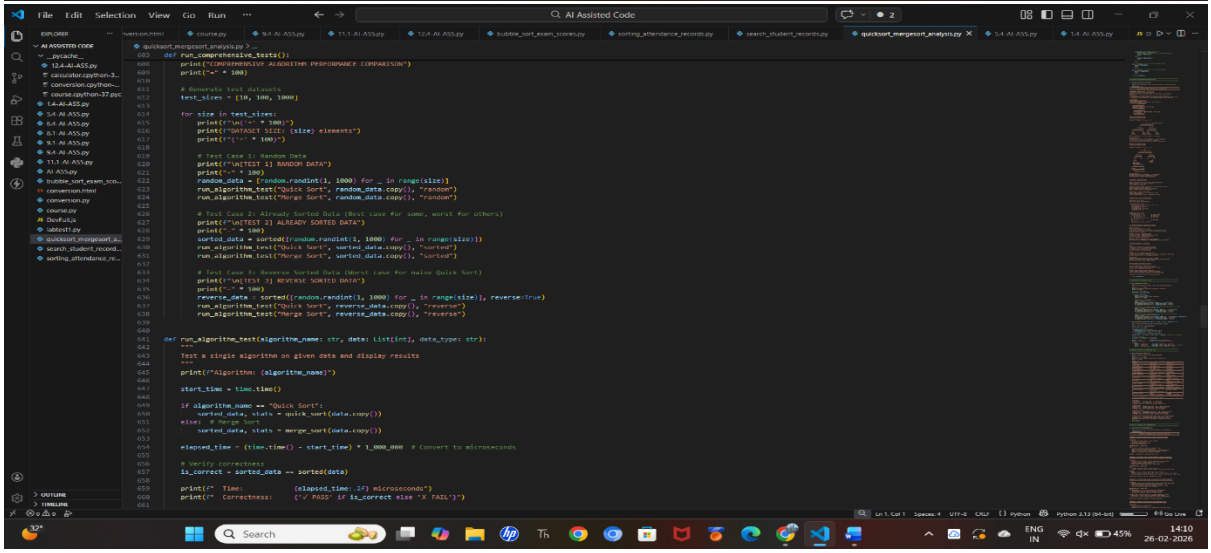
Scenario: You are part of a data analytics team that needs to sort large datasets received in:
Random order & Already sorted order & Reverse sorted order
You must choose between Quick Sort and Merge Sort for better performance.

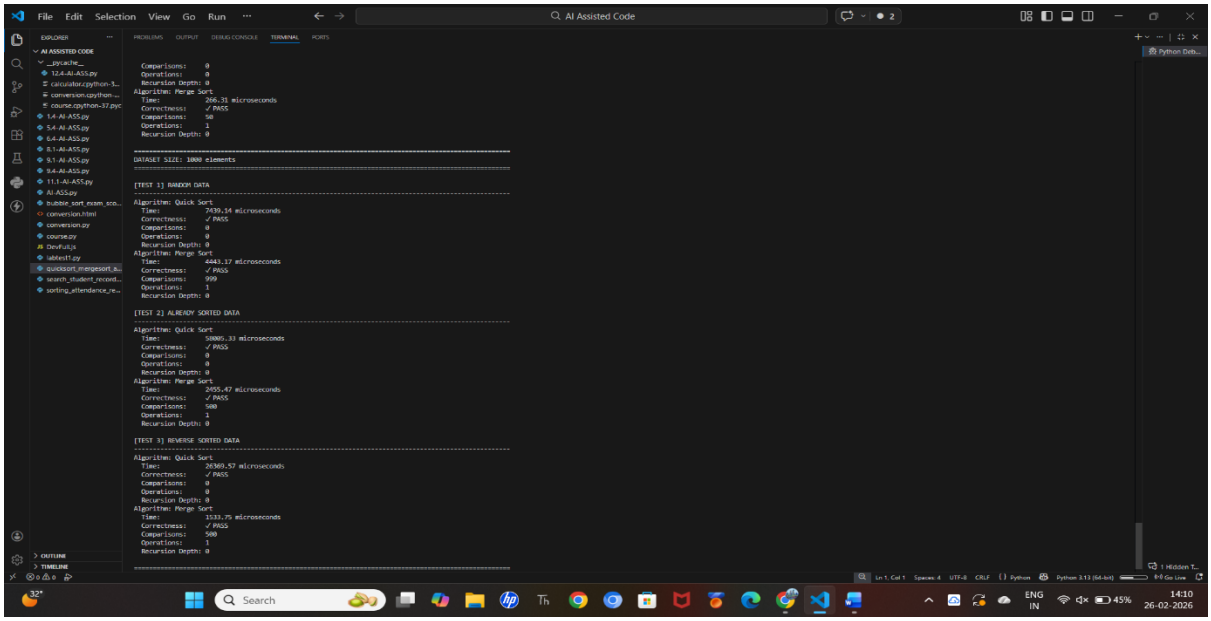
Prompt:

“Complete the recursive logic for Quick Sort and Merge Sort in Python. Add meaningful docstrings explaining parameters and return values. Also explain how recursion works in both algorithms and compare their performance on random, sorted, and reverse-sorted data.”

Code:







Observation:

Random Data: Both performed well (O(n log n)).
Sorted Data: Quick Sort may degrade to O(n²) if pivot is poorly chosen.
Reverse Sorted Data: Same worst-case issue for Quick Sort.
Use Quick Sort for faster average performance and lower memory usage & Use Merge Sort when: Stable sorting is required
Worst-case performance must remain O(n log n) & Handling very large or linked-list data
Merge Sort is more predictable, while Quick Sort is usually faster in practice.

TASK-5

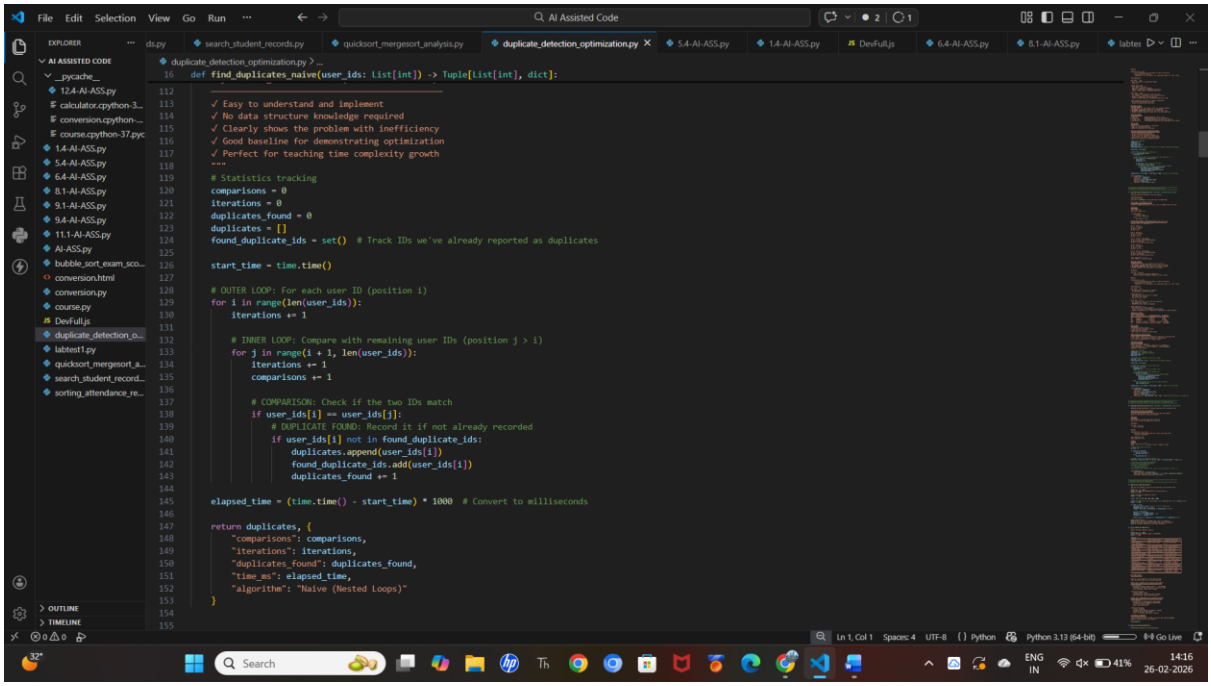
Scenario:

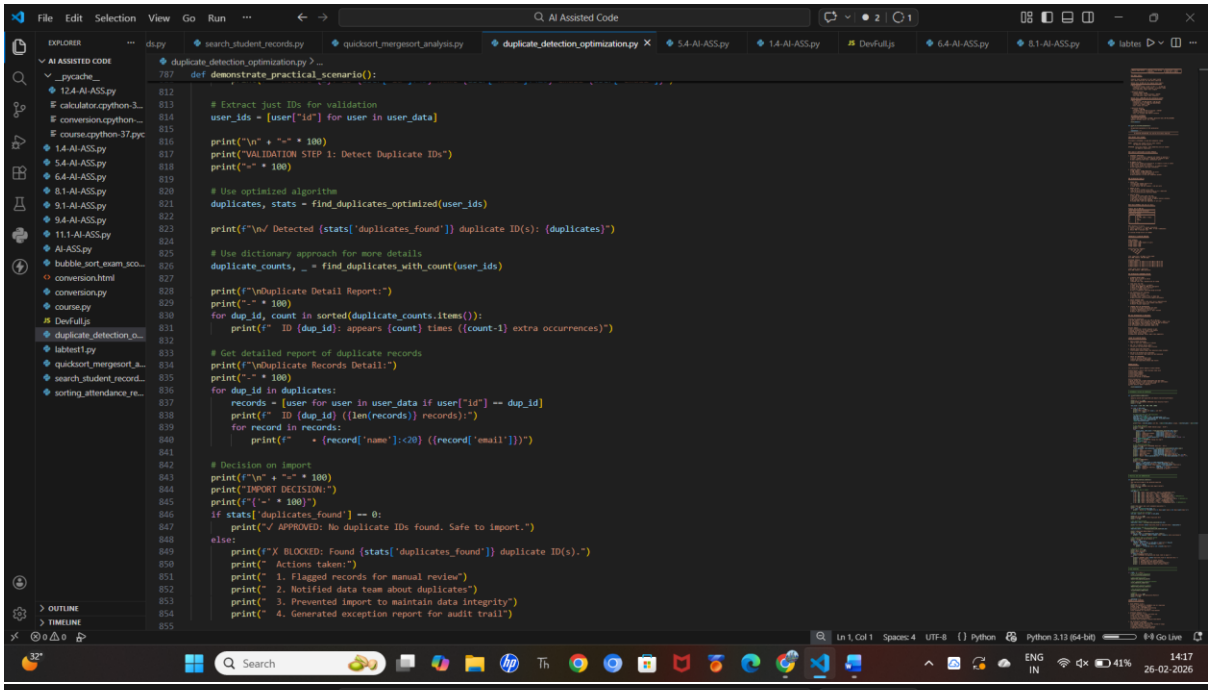
You are building a data validation module that checks for duplicate user IDs in a large dataset before importing it into the system.

Prompt:

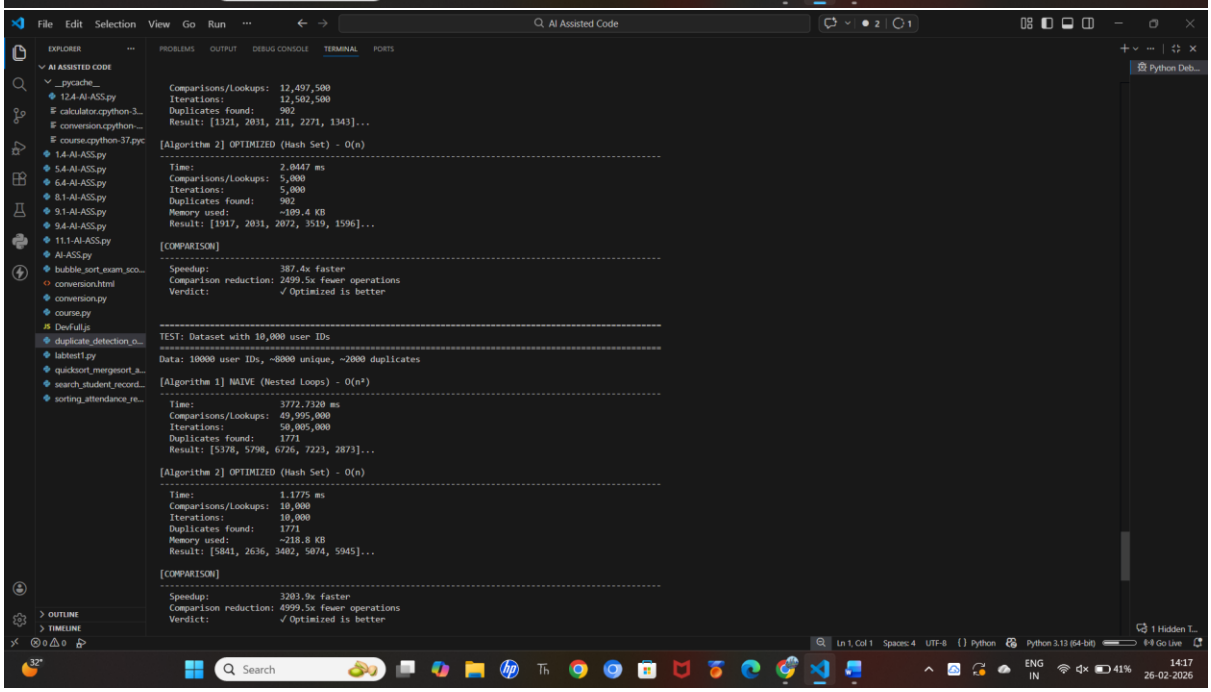
“Write a Python program to detect duplicates using nested loops. Analyze its time complexity. Suggest a more efficient approach using sets or dictionaries. Rewrite the optimized version and compare performance for large inputs.”

Code:





```
    # Extract just IDs for validation
    user_ids = [user["id"] for user in user_data]

    print("\n" + "=" * 100)
    print("VALIDATION STEP 1: Detect Duplicate IDs")
    print("=" * 100)

    # Use optimized algorithm
    duplicates, stats = find_duplicates_optimized(user_ids)

    print(f"\n✓ Detected {stats['duplicates_found']} duplicate ID(s): {duplicates}")

    # Use dictionary approach for more details
    duplicate_counts, _ = find_duplicates_with_count(user_ids)

    print(f"\nDuplicate Detail Report:")
    print("-" * 100)
    for dup_id, count in sorted(duplicate_counts.items()):
        print(f"  ID {dup_id}: appears {count} times ({count-1} extra occurrences)")

    # Get detailed report of duplicate records
    print(f"\nDuplicate Records Detail:")
    print("-" * 100)
    for dup_id in duplicates:
        records = [user for user in user_data if user["id"] == dup_id]
        print(f"  ID {dup_id} ({len(records)} records):")
        for record in records:
            print(f"    • {record['name']:<20} ({record['email']})")

    # Decision on import
    print(f"\n" + "=" * 100)
    print("IMPORT DECISION:")
    print(f"{'-' * 100}")
    if stats['duplicates_found'] == 0:
        print("✓ APPROVED: No duplicate IDs found. Safe to import.")
    else:
        print(f"✗ BLOCKED: Found {stats['duplicates_found']} duplicate ID(s).")
        print("  Actions taken:")
        print("  1. Flagged records for manual review")
        print("  2. Notified data team about duplicates")
        print("  3. Prevented import to maintain data integrity")
        print("  4. Generated exception report for audit trail")
```



```
  Comparisons/Lookups:  12,497,500
  Iterations:           12,502,500
  Duplicates found:     902
  Result: [1321, 2031, 211, 2271, 1343]...

[Algorithm 2] OPTIMIZED (Hash Set) - O(n)
--------------------------------------------------------------------------------
  Time:                 2.0447 ms
  Comparisons/Lookups:  5,000
  Iterations:           5,000
  Duplicates found:     902
  Memory used:          ~109.4 KB
  Result: [1917, 2031, 2072, 3519, 1596]...

[COMPARISON]
--------------------------------------------------------------------------------
  Speedup:              387.4x faster
  Comparison reduction: 2499.5x fewer operations
  Verdict:              ✓ Optimized is better


================================================================================
TEST: Dataset with 10,000 user IDs
================================================================================
Data: 10000 user IDs, ~8000 unique, ~2000 duplicates

[Algorithm 1] NAIVE (Nested Loops) - O(n²)
--------------------------------------------------------------------------------
  Time:                 3772.7320 ms
  Comparisons/Lookups:  49,995,000
  Iterations:           50,005,000
  Duplicates found:     1771
  Result: [5378, 5798, 6726, 7223, 2873]...

[Algorithm 2] OPTIMIZED (Hash Set) - O(n)
--------------------------------------------------------------------------------
  Time:                 1.1775 ms
  Comparisons/Lookups:  10,000
  Iterations:           10,000
  Duplicates found:     1771
  Memory used:          ~218.8 KB
  Result: [5841, 2636, 3402, 5074, 5945]...

[COMPARISON]
--------------------------------------------------------------------------------
  Speedup:              3203.9x faster
  Comparison reduction: 4999.5x fewer operations
  Verdict:              ✓ Optimized is better
```

Observation:

Brute-Force Method:

Uses nested loops to compare every element with others.

Time Complexity: $O(n^2)$

Slow for large datasets because comparisons increase rapidly.

Optimized Method (Using Set):

Traverse list once.

Store seen IDs in a set.

If ID already exists in set → duplicate found.

Time Complexity: O(n)

Much faster for large datasets.